

Standard Operating Procedure (SOP)

Title

Deploying a Custom HTML Website on K3s Using Nginx

Objective

To deploy a static HTML website (Netflix-style UI) on a K3s Kubernetes cluster using an Nginx container and expose it through a Kubernetes Service.

Prerequisites

- Linux server with K3s installed
 - kubectl configured and accessible
 - Internet connectivity to pull container images
 - Basic knowledge of Kubernetes resources
-

Environment Details

Component	Value
Kubernetes Distribution	K3s
Web Server	Nginx
Container Image	nginx:latest
Application Type	Static HTML Website
Service Type	NodePort

Procedure

Step 1: Create the HTML File

Create an `index.html` file containing the Netflix-style UI.

Example:

```
<h1>Today Trending</h1>
<p>Watch the most popular shows and movies trending today.</p>
```

Store the file on the local system.

Step 2: Create a ConfigMap

Create a ConfigMap from the HTML file:

```
kubectl create configmap nginx-html
--from-file=index.html
```

Verify:

```
kubectl get configmap
```

Expected Output:

```
nginx-html
```

Step 3: Create Deployment

Create a deployment YAML file.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-web
  template:
    metadata:
      labels:
        app: nginx-web
    spec:
      containers:
```

```
- name: nginx
  image: nginx:latest
  ports:
  - containerPort: 80
  volumeMounts:
  - name: html-volume
    mountPath: /usr/share/nginx/html
  volumes:
  - name: html-volume
    configMap:
      name: nginx-html
```

Apply deployment:

```
kubectl apply -f deployment.yaml
```

Verify:

```
kubectl get pods
```

Expected:

```
nginx-web-xxxxxxxx-xxxxx 1/1 Running
```

Step 4: Expose Deployment

Create a NodePort service:

```
kubectl expose deployment nginx-web
--type=NodePort
--port=80
```

Verify:

```
kubectl get svc
```

Example Output:

NAME	TYPE	CLUSTER-IP	PORT(S)
nginx-web	NodePort	10.43.22.100	80:31893/TCP

Step 5: Access the Website

Obtain the node IP address:

```
ip addr
```

Example:

```
172.16.3.63
```

Access the application:

```
http://172.16.3.63:31893
```

Step 6: Verify Deployment

Check pod status:

```
kubectl get pods
```

Check service:

```
kubectl get svc
```

Check deployment:

```
kubectl get deployment
```

Troubleshooting

Pod Not Running

Check pod details:

```
kubectl describe pod <pod-name>
```

Check logs:

```
kubectl logs <pod-name>
```

Website Not Accessible

Verify service:

```
kubectl get svc
```

Verify node port is open:

```
ss -tulnp | grep <nodeport>
```

HTML Changes Not Reflected

Restart deployment:

```
kubectl rollout restart deployment nginx-web
```

Verify mounted files:

```
kubectl exec -it <pod-name> -- ls /usr/share/nginx/html
```

Validation Checklist

- [] K3s cluster operational

- [] ConfigMap created successfully
 - [] Nginx deployment running
 - [] Service exposed using NodePort
 - [] Website accessible from browser
 - [] Custom HTML page displayed correctly
-

Commands Summary

```
kubectl create configmap nginx-html --from-file=index.html

kubectl apply -f deployment.yaml

kubectl expose deployment nginx-web --type=NodePort --port=80

kubectl get pods

kubectl get svc

kubectl get deployments

kubectl rollout restart deployment nginx-web
```

Result

A static Netflix-style HTML website was successfully deployed on a K3s Kubernetes cluster using Nginx and exposed through a NodePort service, allowing access through a web browser using the node IP and assigned NodePort.